

Pickle Stability & Correctness

Test Suite — Final Report

Agenda

01 Introduction — What is pickle?

02 Research Question

03 Testing Strategies

04 Work Distribution

05 Results — 80/80 Passed

06 Findings

07 Limitations

08 Conclusion

01 Introduction

What is pickle?

A Python standard-library module that converts objects into **byte streams** (serialization) and back again (deserialization).

Pickling → object → bytes

Unpickling → bytes → object

Research Question

Does the same input always produce the same **hash-identical** output?

Scenarios to test:

- Different operating systems
- Different Python versions
- Floating-point precision
- Recursive data structures

03 Testing Strategies

Equivalence Partitioning

27 tests

None, bool, int, float,
str, bytes, list, tuple,
dict, set, custom class

Boundary Value Analysis

25 tests

sys.maxsize, 10^{1000} ,
 ± 0.0 , NaN, $\pm \text{inf}$,
string 255/256/65535

Fuzzing

20 subtests

20 fixed seeds —
random nested objects
(reproducible)

White-box Testing

5 tests

Memo dict, reduce_ex,
getstate/setstate,
PROTO opcode

+ Round-trip correctness (9 tests) + Protocol versions 0–5 (8 tests) + dumps vs dump-to-file (1 test)

04 Work Distribution

Member 1

CODE:

- TestEquivalencePartitioning (27 tests)
- TestBoundaryValues (25 tests)
- TestRoundTrip (9 tests)
- Helper utilities (pickle_sha256, assert_hash_identical)

REPORT:

Sections 1–3 (Introduction, Test Suite Overview, Strategies 3.1 & 3.2)
Traceability matrix rows 1–8

Member 2

CODE:

- TestRecursiveStructures (5 tests)
- TestProtocolVersions (8 tests)
- TestDumpVsFile (1 test)
- TestFuzzing (20 subtests)
- TestWhiteBox (5 tests)

REPORT:

Sections 4–7 (Findings, Traceability rows 9–15, Limitations, Conclusion)
GitHub repository management

05 Results

80

PASSED

0 failed • 0.16 seconds

Python 3.9 / 3.10 • Linux & Windows

27

Equivalence
Partitioning

25

Boundary
Value

20

Fuzzing
subtests

8

Protocol
tests

Terminal

```
TestWhiteBox::test_memo_deduplication
```

```
PASSED [96%]
```

```
TestWhiteBox::test_proto_opcode
```

```
PASSED [98%]
```

```
TestWhiteBox::test_reduce_ex_dispatch
```

```
PASSED [100%]
```

```
———— 80 passed in 0.16s ————
```

06 Findings

✓ Stable

Primitive types — all deterministic ✓

Float specials: NaN, $\pm\infty$, -0.0 ✓

Very large integers (10^{1000}) ✓

Unicode strings up to 65,535 chars ✓

Self-referential list / dict ✓

Custom class (`__slots__`, `__getstate__`) ✓

Protocol 0–5 each deterministic ✓

`dumps()` $\equiv$ `dump()-to-BytesIO` ✓

⚠ Note

set / frozenset — may be non-deterministic
due to hash randomisation (PYTHONHASHSEED)

Different protocols produce different bytes
(expected and documented behaviour)

07 Limitations

No cross-process stability test

Tests run within a single process. True stability requires comparing pickle files from separate Python invocations on different OSes.

Hash randomisation not toggled

set/frozenset stability requires launching separate processes with fixed vs. random PYTHONHASHSEED.

No cross-version testing

Only tested on Python 3.9 & 3.10. Versions 3.8 and 3.11 require a CI matrix (e.g. GitHub Actions tox).

C extension coverage gap

coverage.py cannot instrument the `__pickle` C module — branch coverage of the fast path is unknown.

datetime, decimal, enum not tested

These stdlib types have custom `__reduce__` implementations that were not included in the current suite.

Conclusion

- 1 80/80 tests passed — pickle is deterministic within a single process
- 2 set / frozenset can be non-deterministic due to hash randomisation
- 3 Different protocol versions produce different byte streams (expected)
- 4 Future work: cross-process, cross-version, and cross-OS testing via CI